ELSEWEDY UNIVERSITY OF TECHNOLOGY
- POLYTECHNIC OF EGYPT -

CET322

Lecture 4: Containers

Outline

- Motivation and overview
- Linux container techniques
- Docker

Acknowledgment: some slides from “Introduction to Docker” (Docker Inc.)

Do VMs Fit All Today's Cloud User Needs?

- Performance overhead of indirections (guest OS and hypervisor)
- Large memory footprint
- Slow startup time
- License and maintenance cost of guest OS

- Do we really need to virtualize hardware and a full OS?

Recent Trend: Container

- Light-weight virtualization
  - Running multiple isolated user-space applications on one OS
  - Virtualization layer runs as an application within the OS
  - Focusing on resource and namespace isolation

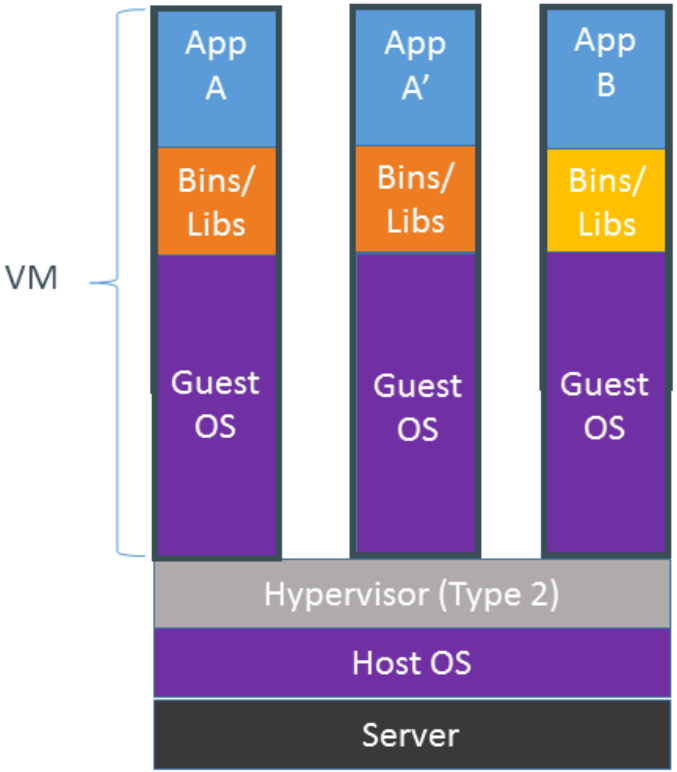
- Example: Docker, LXC, Kubernetes, Xen Unikernel

Micro Machines for Microservices

- Containers:
  - Containers typically contain a single app or microservice, alongside the various runtime libraries they need to run. Importantly, they do not contain an operating system, and they do not need a hypervisor to run – they run on a shared engine kernel, thus effectively operating on bare metal.

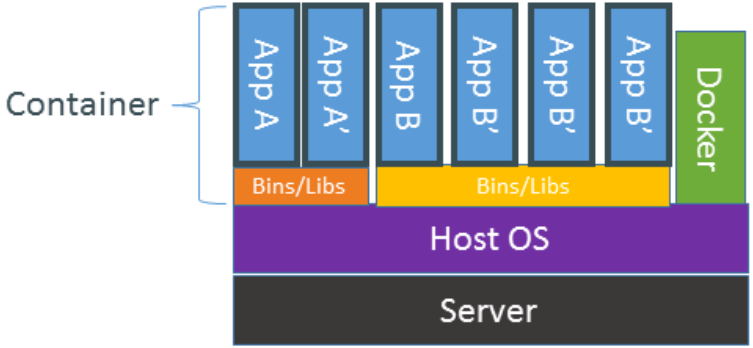
- Virtual machines:
  - Virtual machines images are also a good choice for when your applications and services need to run across multiple operating systems in the data center. They are not restricted to a single OS kernel in the way that containers are.

VM vs Container

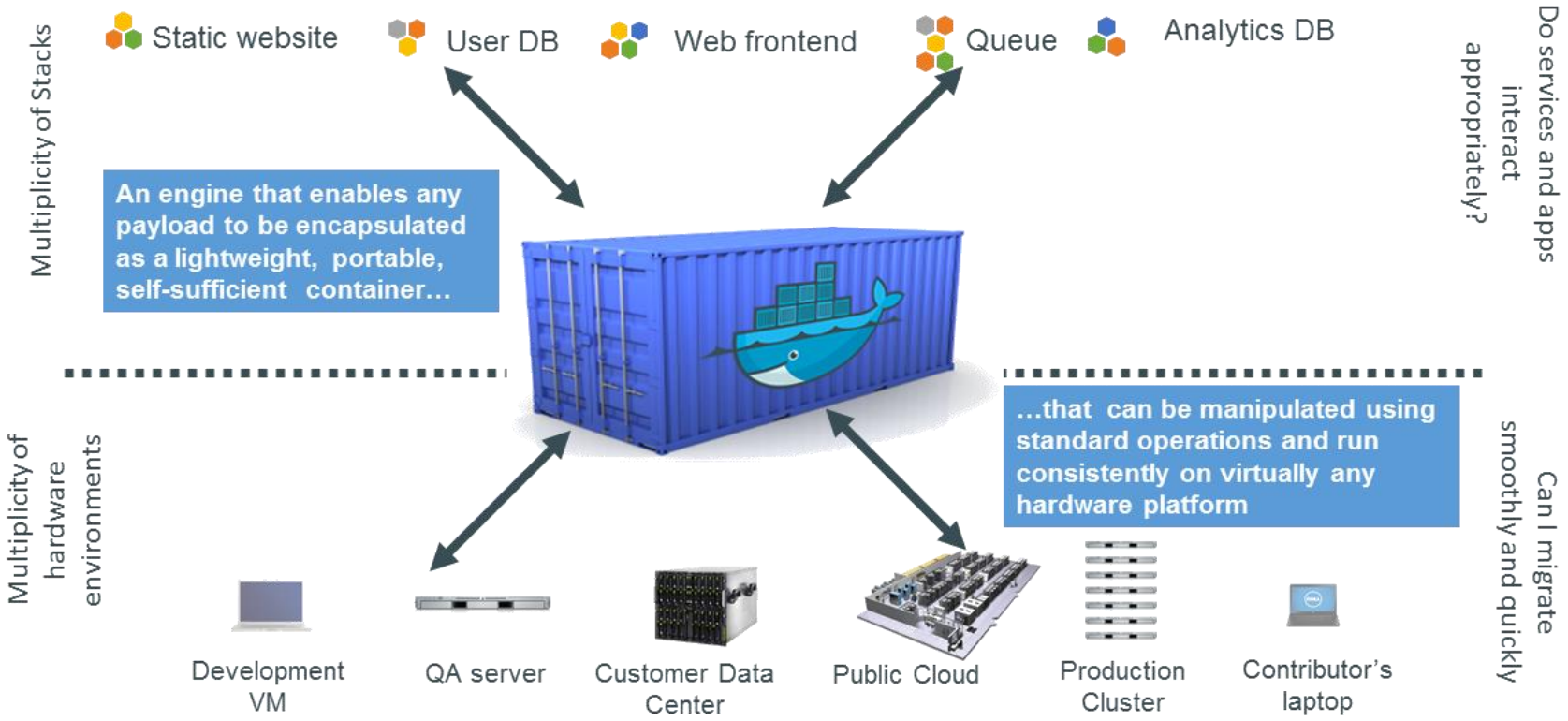


Containers are isolated, but share OS and, where appropriate, bins/libraries

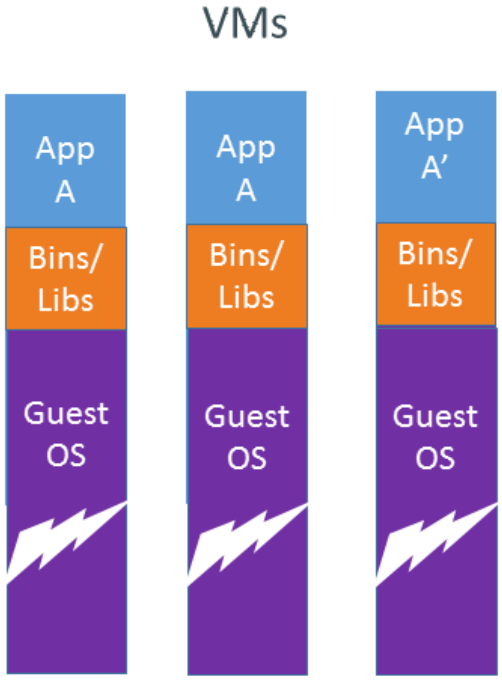
...result is significantly faster deployment, much less overhead, easier migration, faster restart



Docker: Container for Code

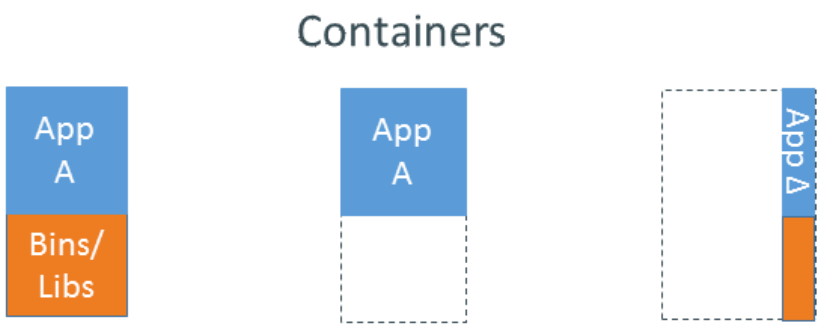


Why are Containers Lightweight?



VMs

Every app, every copy of an app, and every slight modification of the app requires a new virtual server



Original App

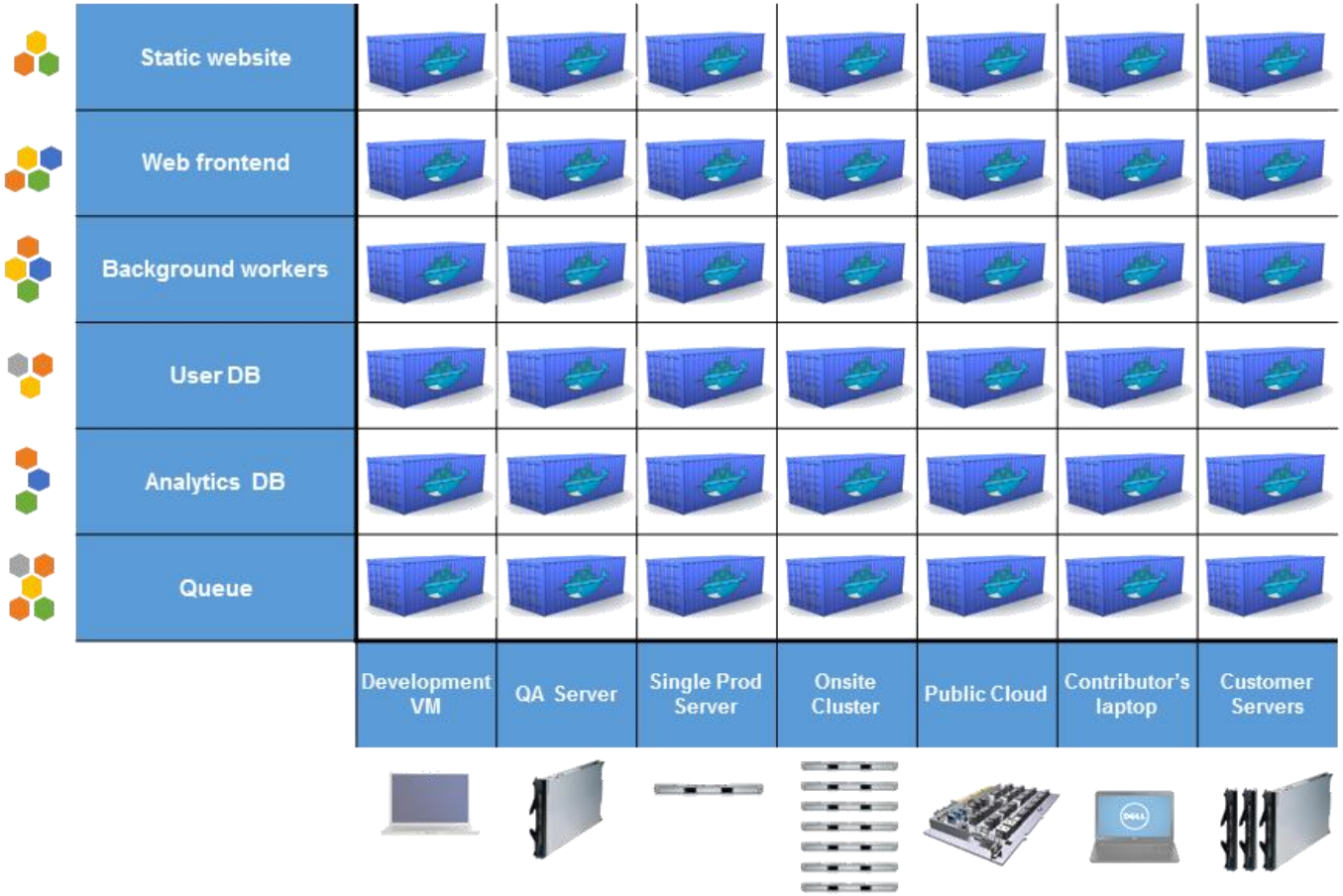
(No OS to take up space, resources, or require restart)

Copy of App

No OS. Can Share bins/libs

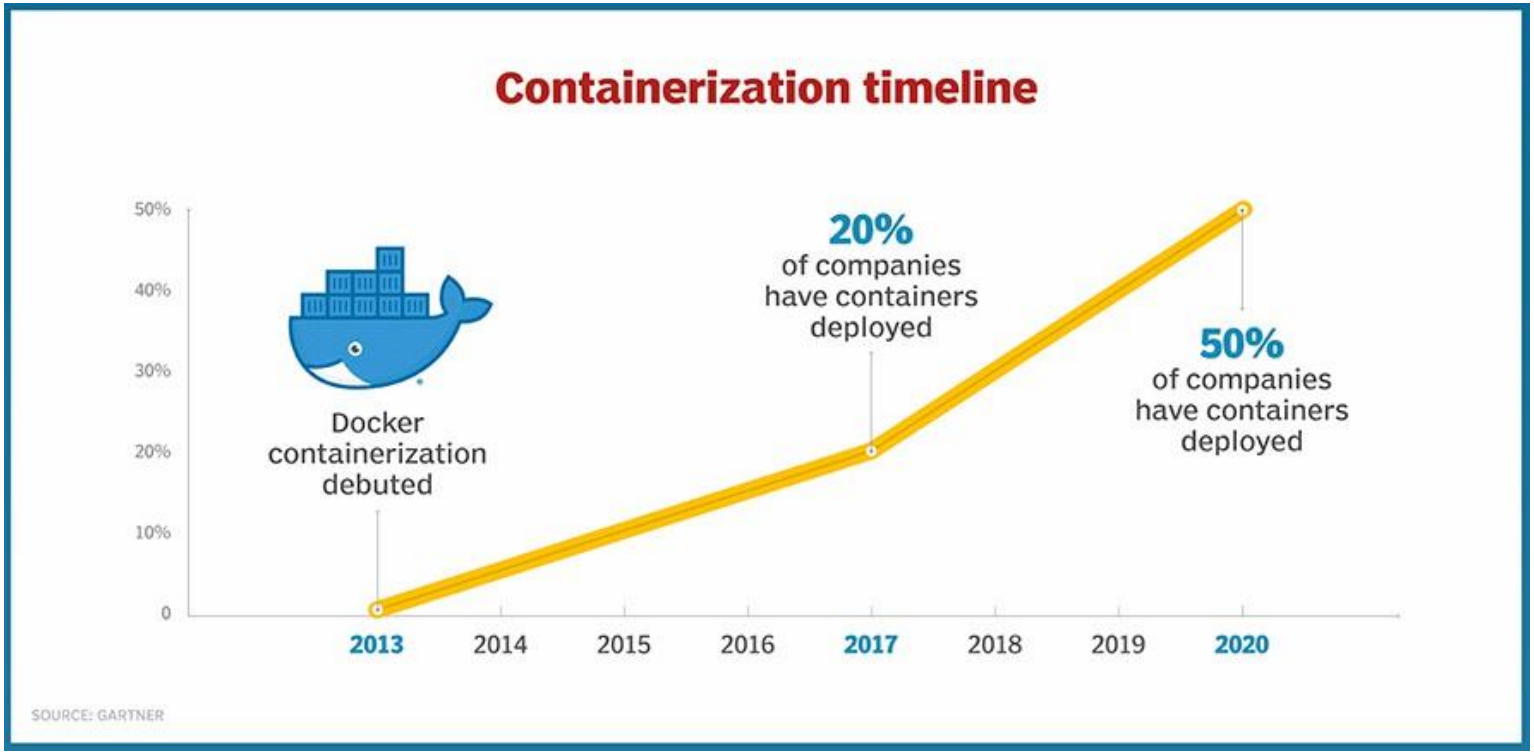
Modified App

Copy on write capabilities allow us to only save the diffs Between container A and container A'


Static website
Web frontend
Background workers
User DB
Analytics DB
Queue
Development VM
QA Server
Single Prod Server
Onsite Cluster
Public Cloud
Contributor's laptop
Customer Servers

Efficiency: *almost* no overhead

- Processes are isolated, but run straight on the host
- CPU performance = native performance
- Memory performance = a few % shaved off for (optional) accounting
- Network performance = small overhead; can be optimized to zero overhead


Containerization timeline
50%
40%
30%
20%
10%
0
Docker containerization debuted
20% of companies have containers deployed
50% of companies have containers deployed
2013
2014
2015
2016
2017
2018
2019
2020
SOURCE: GARTNER

Linux Containers

- Run everywhere
  - Regardless of kernel version
  - Regardless of host distro
  - Physical or virtual, cloud or not
  - Container and host architecture must match...
- Run anything
  - If it can run on the host, it can run in the container
  - If it can run on a Linux kernel, it can run

At High-Level: It Looks Like a VM

- Own process space
- Own network interface
- Can run stuff as root
- Can have its own /sbin/init (different from the host)

At Low-Level: OS-Level Virtualization

- Containers run on a host OS directly (and share the OS)
- Run as processes
- OS provides resource isolation and namespace isolation

Namespaces and Cgroups

- Two mechanisms in Linux kernel over which containers are built:
  - Namespaces: a way to provide isolated view of a certain global resource (e.g., root filesystem) to a set of processes. Processes within a namespace see only their slice of the global resource
  - Cgroups: a way to set resource limits on a group of processes
- Together, namespaces and cgroups allow us to isolate a set of processes into a bubble and set resource limits
- Container implementations like LXC, Docker leverage these mechanisms to build the container abstractions
  - LXC is general container while Docker optimized for single application
- Frameworks like Docker Swarm or Kubernetes help manage multiple containers across hosts, along with autoscaling, lifecycle management, and so on

Namespaces in operation (lwn.net) https://lwn.net/Articles/531114/

Documentation/cgroups/cgroups.txt https://lwn.net/Articles/524935/

Isolating Resources with cgroups

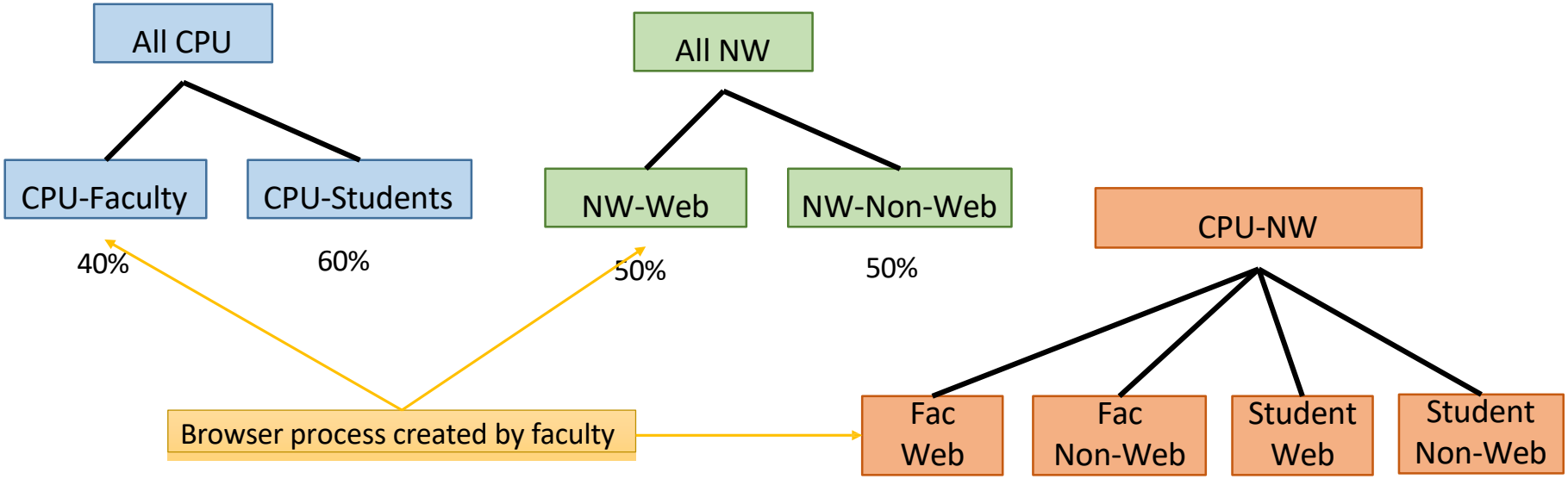
- Linux Control Groups (cgroups): collection of Linux processes
  - Limits resource usages at group level (e.g., memory, CPU, device)
  - Fair sharing of resources
  - Track resource utilization (e.g., could be used for billing/management)
  - Control processes (e.g., pause/resume, checkpoint/restore)

The next building block: Cgroups

- Namespaces let us isolate processes into a slice with respect to many resources: mount points, PID/UID numberspace, network endpoints, etc.
- And Cgroups will let us assign resource limits on a set of processes
  - Divide processes into groups and subgroups hierarchically
  - Assign resource limits for processes in each group/subgroup
- Which resources can be limited? CPU, memory, I/O, CPU sets (which process can execute on which CPU core), and so on
  - Specify what fraction of resource can be used by each group of processes

Cgroups hierarchies

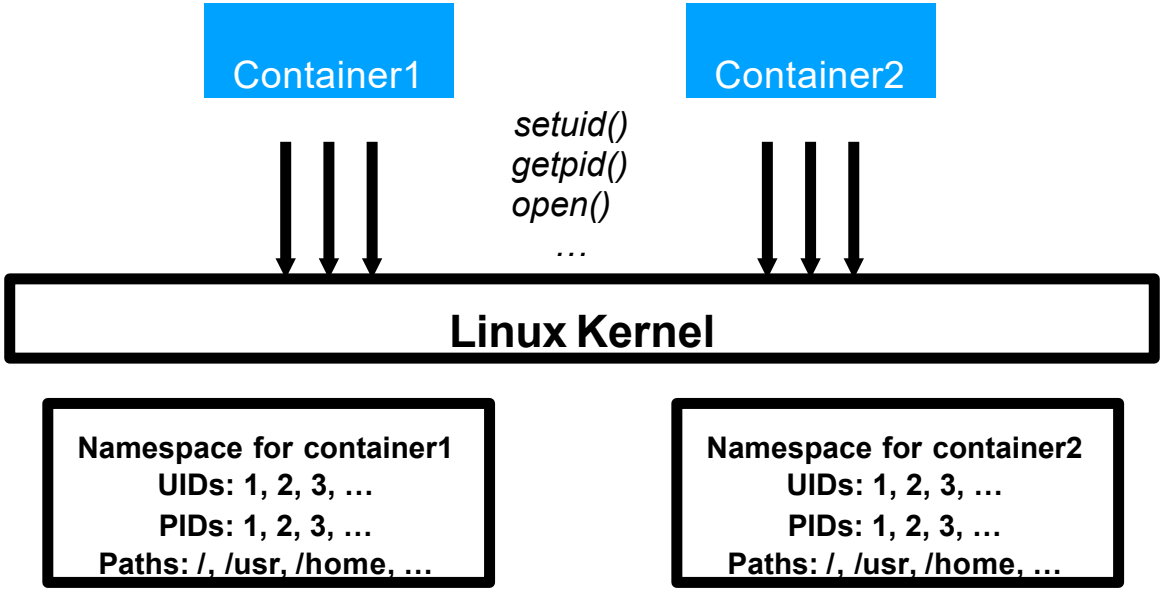
- Can create separate hierarchies for each resource, or a combined hierarchy for multiple resources together



Using Namespaces to Separate “Views” of Users

- Namespace: naming domain for various resources
  - User IDs (UIDs)
  - Process IDs (PIDs)
  - File paths (mnt)
  - Network sockets
  - Pipe names

Namespaces Isolated by Kernel



Namespaces

- Group of processes that have an isolated/sliced view of a global resource
- Default namespace for all processes in Linux, system calls to create new namespaces and place processes in them
- Which resources can be sliced?
    1. Mount namespace: isolates the filesystem mount points seen by a group of processes. The mount() and umount() system calls only affect the processes in that namespace.
    2. PID namespace: isolates the PID numberspace seen by processes. E.g., first process in a new PID namespace gets a PID of 1.
    3. Network namespace: isolates network resources like IP addresses, routing tables, port numbers and so on. E.g., processes in different network namespaces can reuse the same port numbers.
    4. UTS namespace: isolates the hostname seen by processes.
    5. User namespace: isolates the UID/GID numberspace. E.g., a process can get UID=0 (i.e., act as root) in one namespace, while being unprivileged in another namespace. Mappings to be specified between UIDs in parent namespace and UIDs in new namespace.
    6. IPC namespace: isolates IPC endpoints like POSIX message queues.
- More powerful than chroot() which only isolates root filesystem

Namespaces API

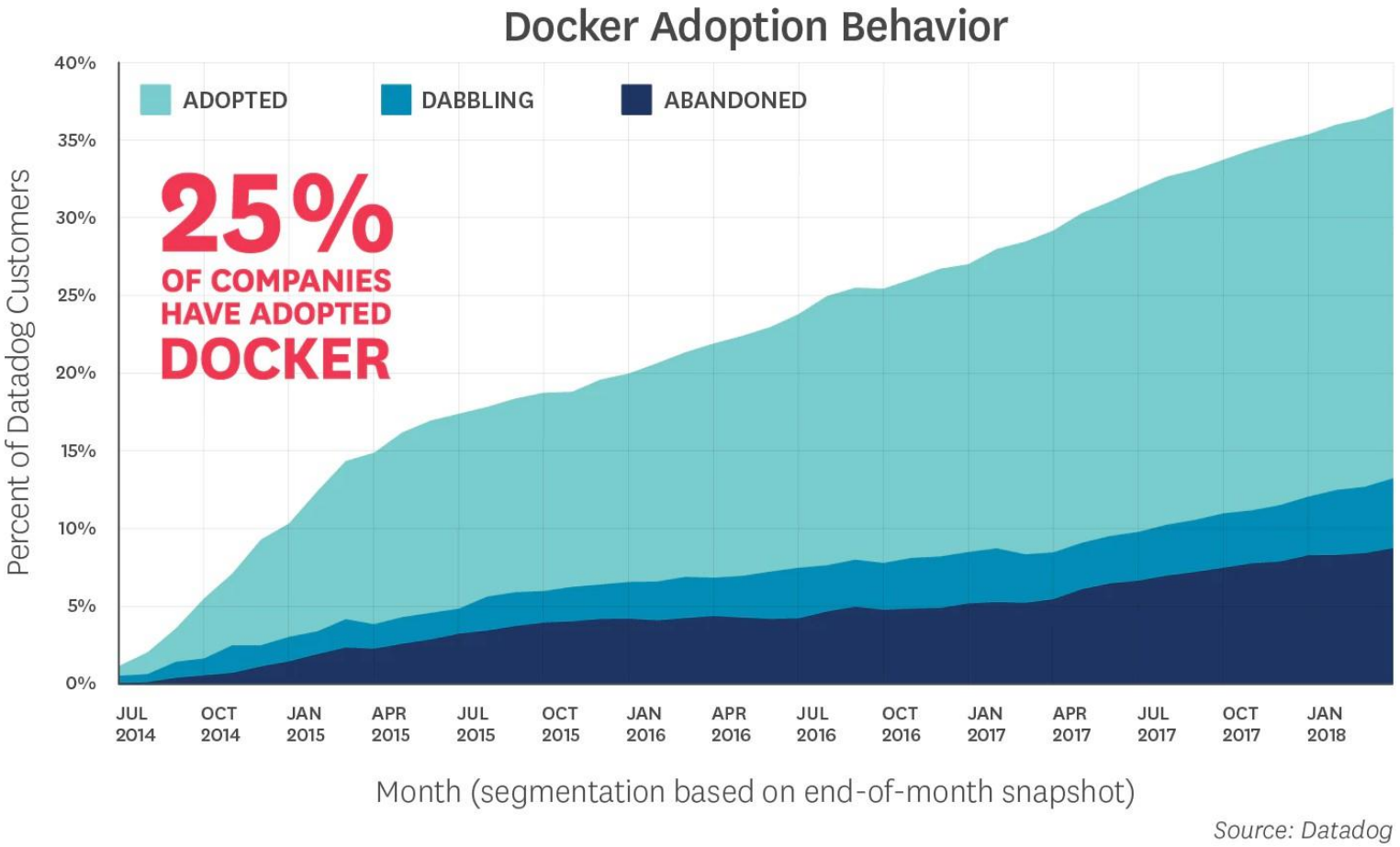
- Three system calls related to namespaces:
  - clone() is used to create a new process and place it into a new namespace. More general version of fork().

```
childPID = clone(childFunc, childStack, flags, arg)
```

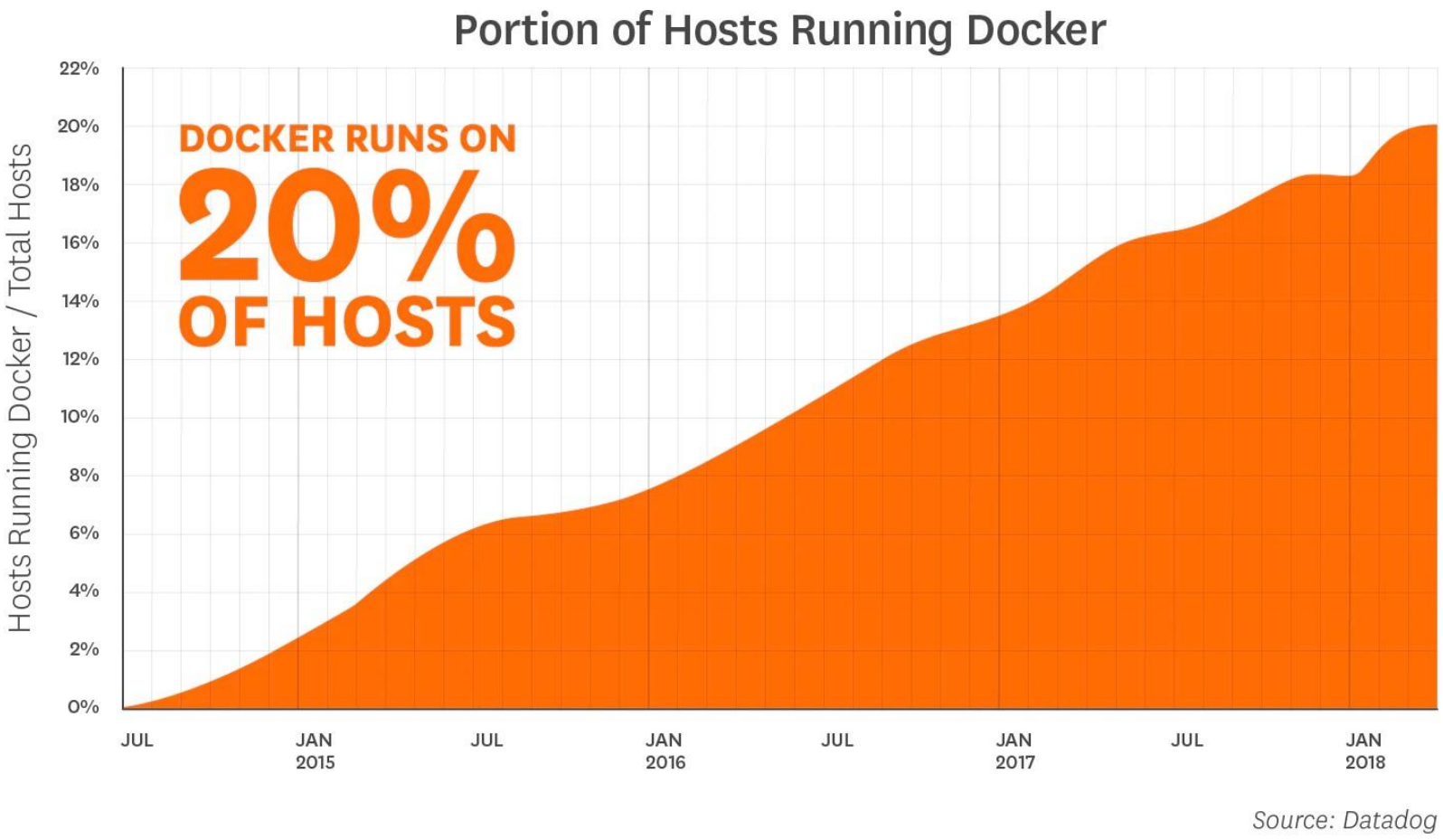
    Flags specify what should be shared with parent, and what should be created new for child (including virtual memory, file descriptors, namespaces etc.)
  - setns() lets a process join an existing namespace. Arguments specify which namespace, and which type.
  - unshare() creates a new namespace and places calling process into it. Flags indicate which namespace to create. Forking a process and calling unshare() is equivalent to clone().
- Once a process is in a namespace, it can open a shell and do other useful things in that namespace
- Forked children of a process belong to parent namespace by default

Docker

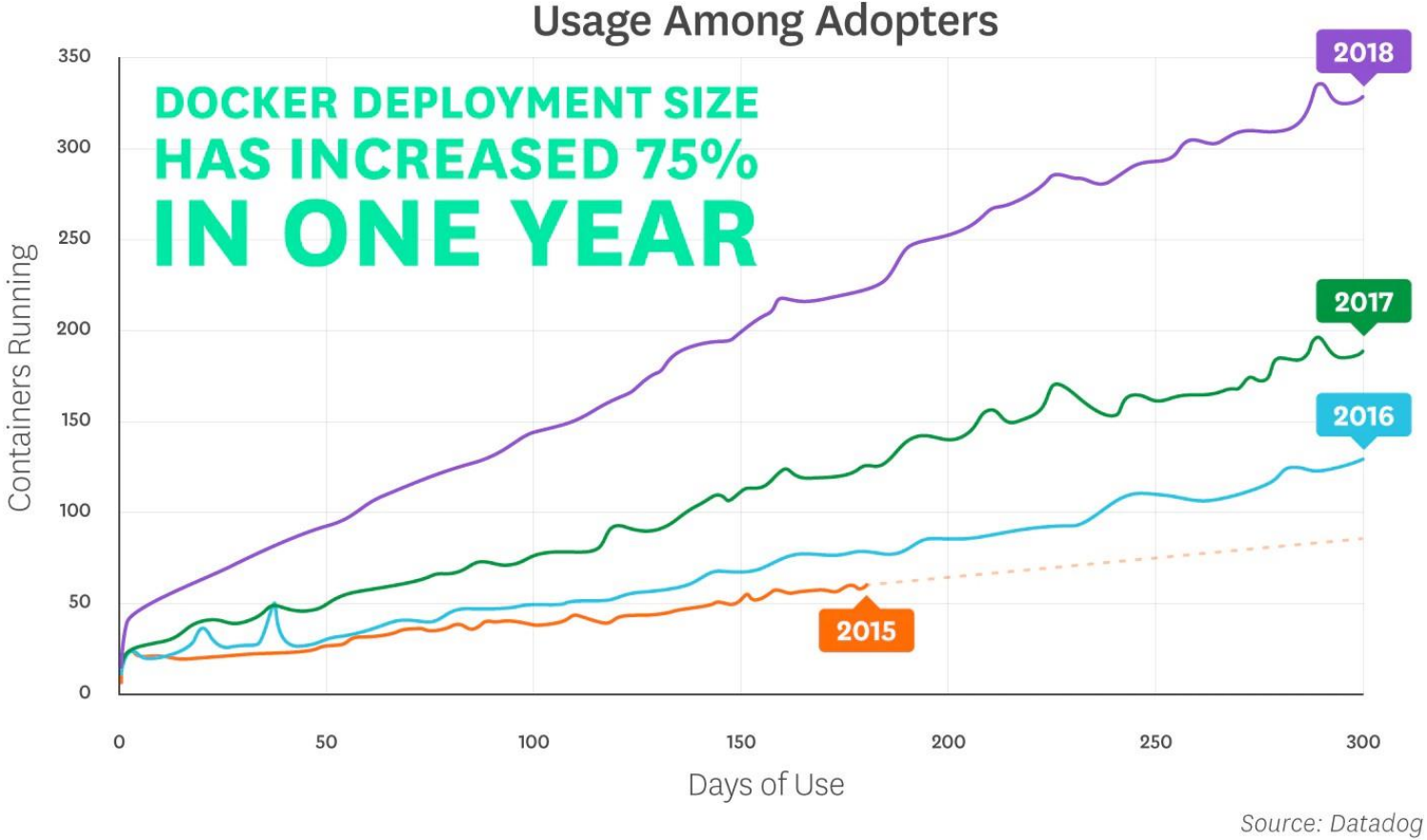
- Docker Inc
  - Founded as dotCloud, Inc. in 2010 by Solomon Hykes (renamed to Docker Inc. in 2013)
  - Estimated to be valued at over $1 billion (101-250 employees)
- Docker the software
  - A container engine written in Go (based on Linux container)
- Docker community
  - Now 2243 contributors, 19K forks of docker engine on GitHub (called Moby)



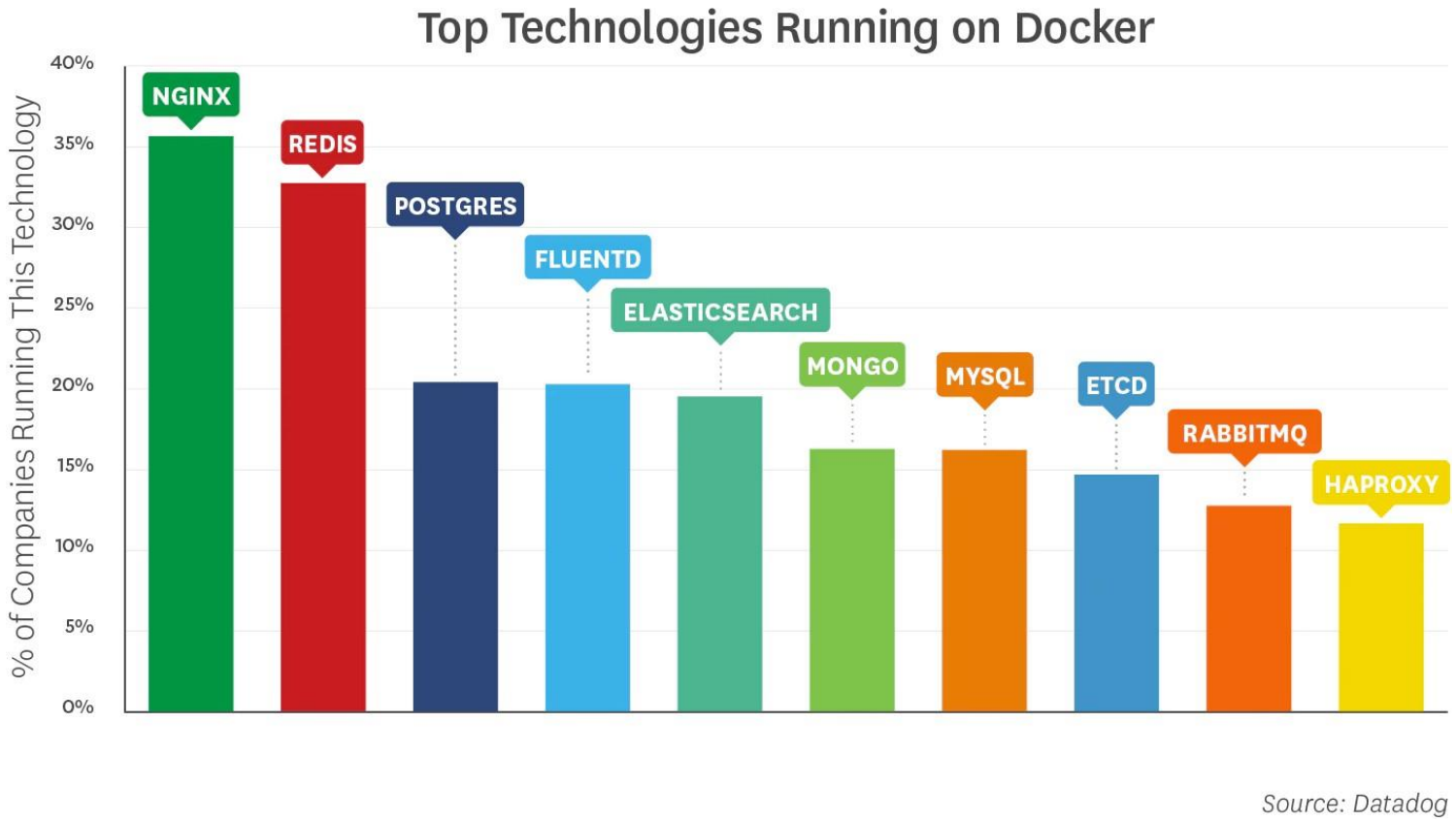
Source: Datadog



Source: Datadog



Source: Datadog



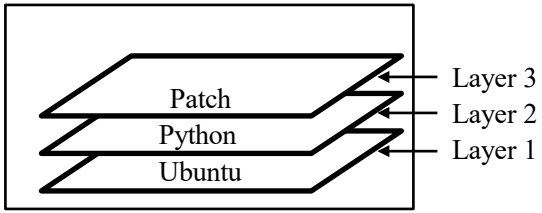
Source: Datadog

Docker Commands

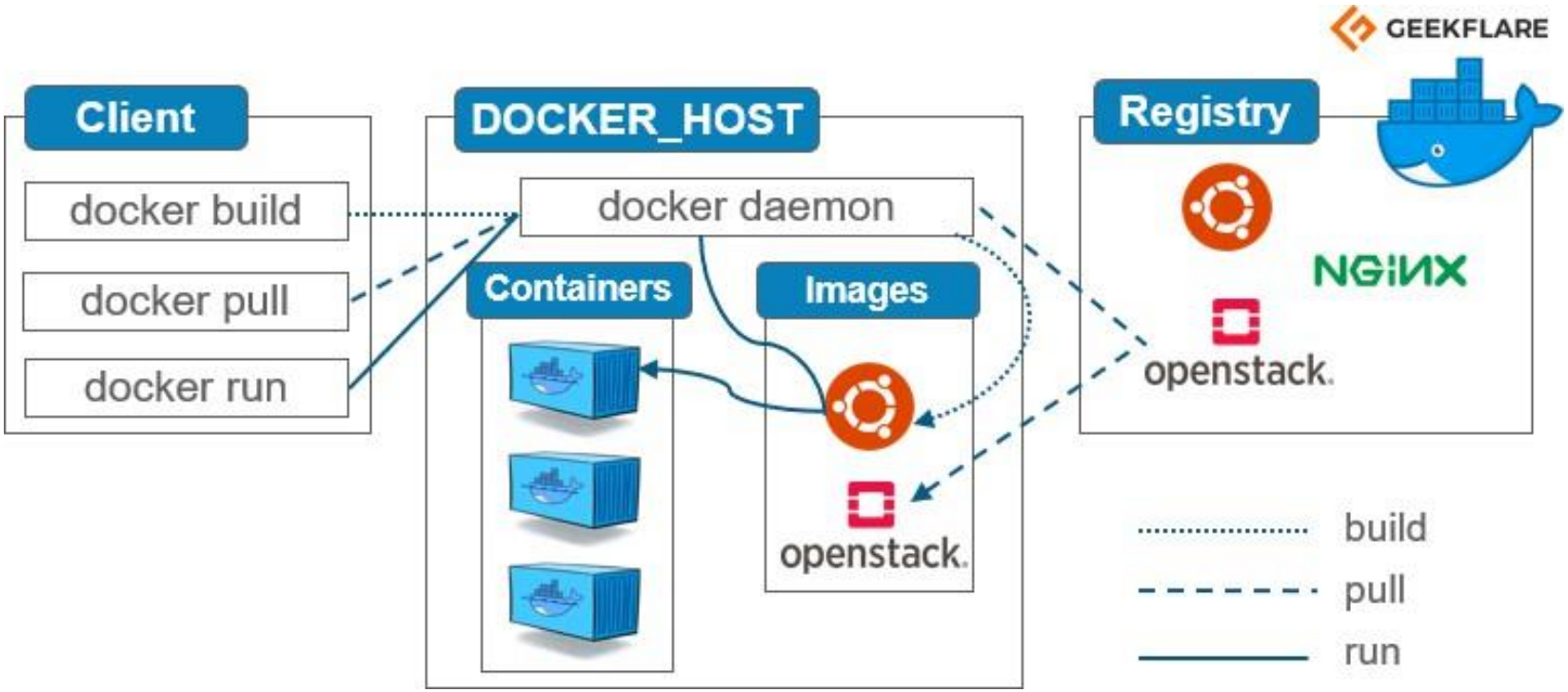
- **docker container run**: Run the specified image
- **docker container ls**: list running containers
- **docker container exec**: run a new process inside a container
- **docker container stop**: Stop a container
- **docker container start**: Start a stopped container
- **docker container rm**: Delete a container
- **docker container inspect**: Show information about a container

Layers

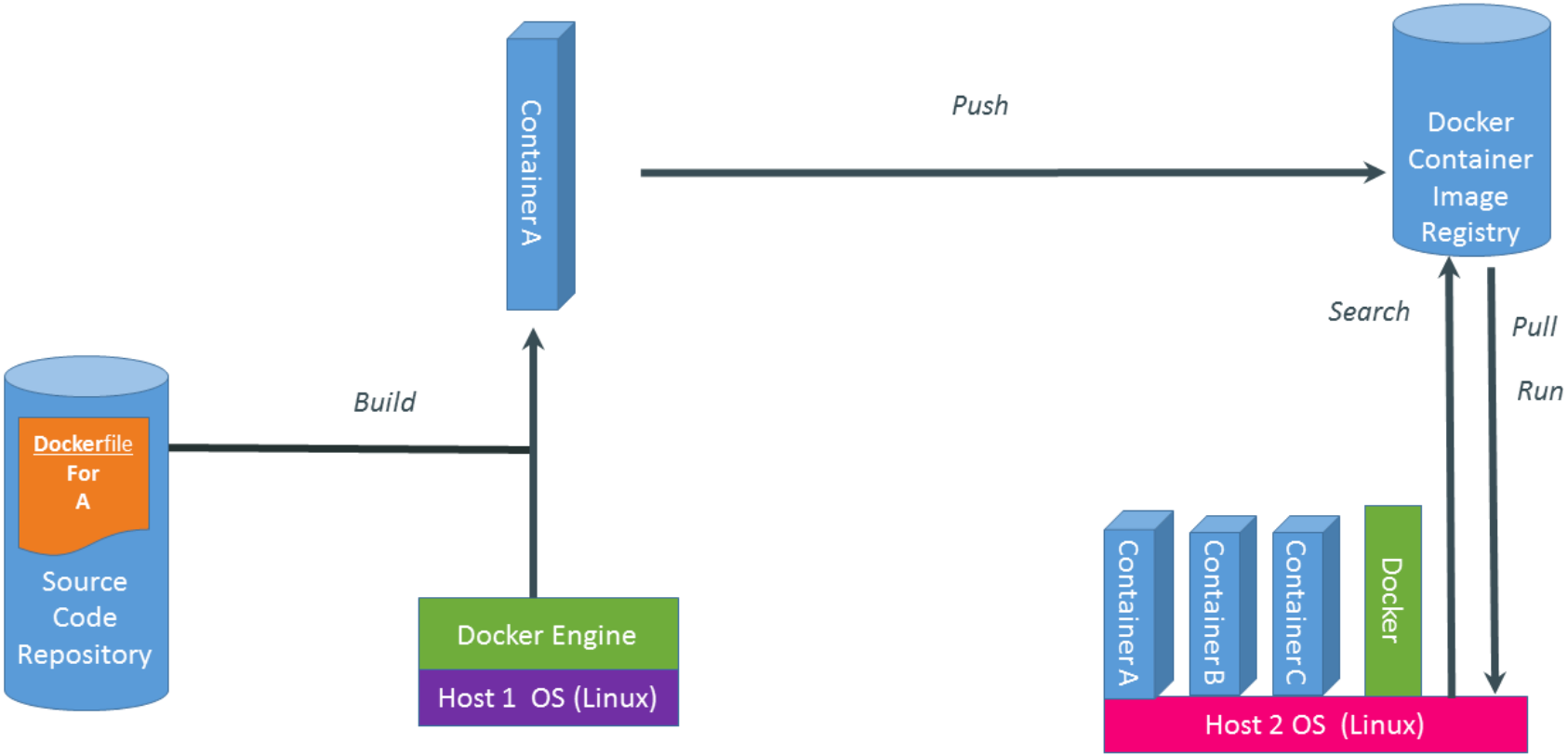
- Each image has many layers
- Image is built layer by layer
- Layers in an image can be inspected by Docker commands
- Each layer has its own 256-bit hash
- For example:
  - Ubuntu OS is installed, then
  - Python package is installed, then
  - a security patch to the Python is installed
- Layers can be shared among many containers



Docker Architecture

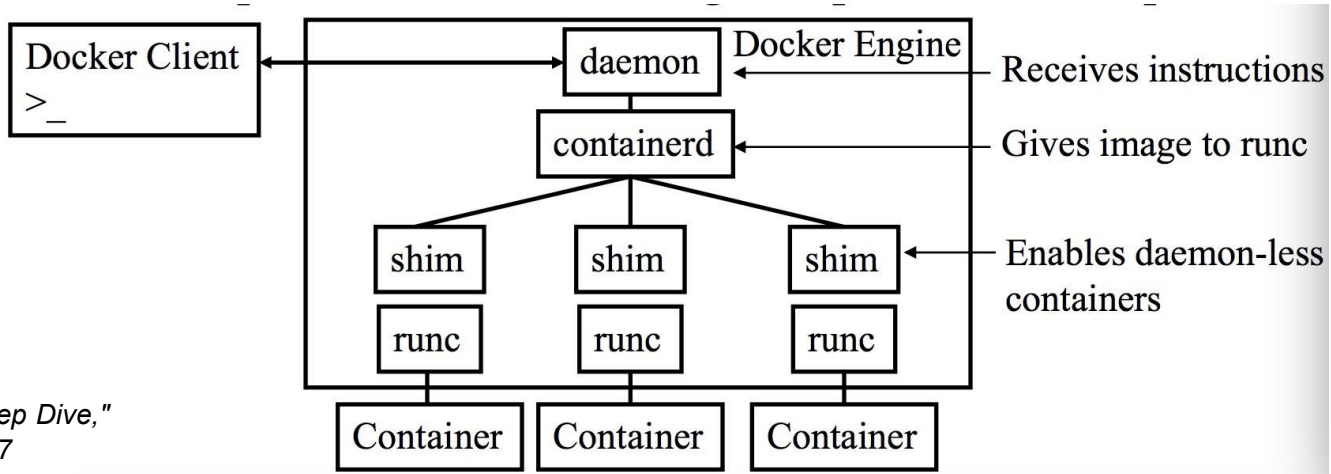


What are the Basics of a Docker System?



Docker Engine

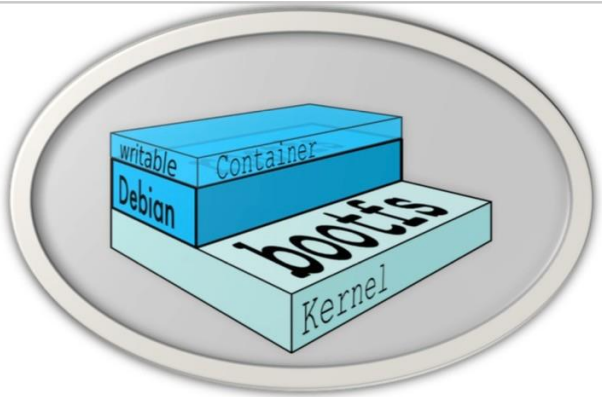
- daemon: Rest API (receiving instructions) and other features
- containerd: Execution logic (e.g., start, stop, pause, unpause, delete containers)
- runc: A lightweight runtime CLI

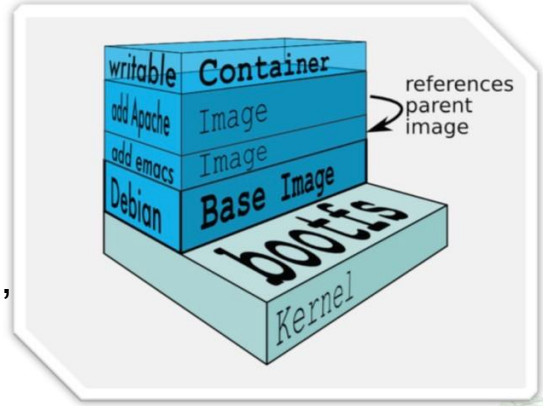


Source: N. Poulton, "Docker Deep Dive," Oct 2017, ISBN: 9781521822807

Docker Images

- Not a VHD, not a file system (Not a VM disk and not a full OS. It doesn't carry a kernel; it relies on the host's kernel).
- uses a Union File System to "overlay" (Images are a stack of read-only layers).
- A read-only Layer, (the image doesn't change at runtime—the container does).
- Do not have state, (Images are stateless, containers hold state).
- Basically a tar file
- Has hierarchy (arbitrary depth).
- An image is the immutable blueprint; a container is a live, isolated process using that blueprint with its own (ephemeral) writable layer.

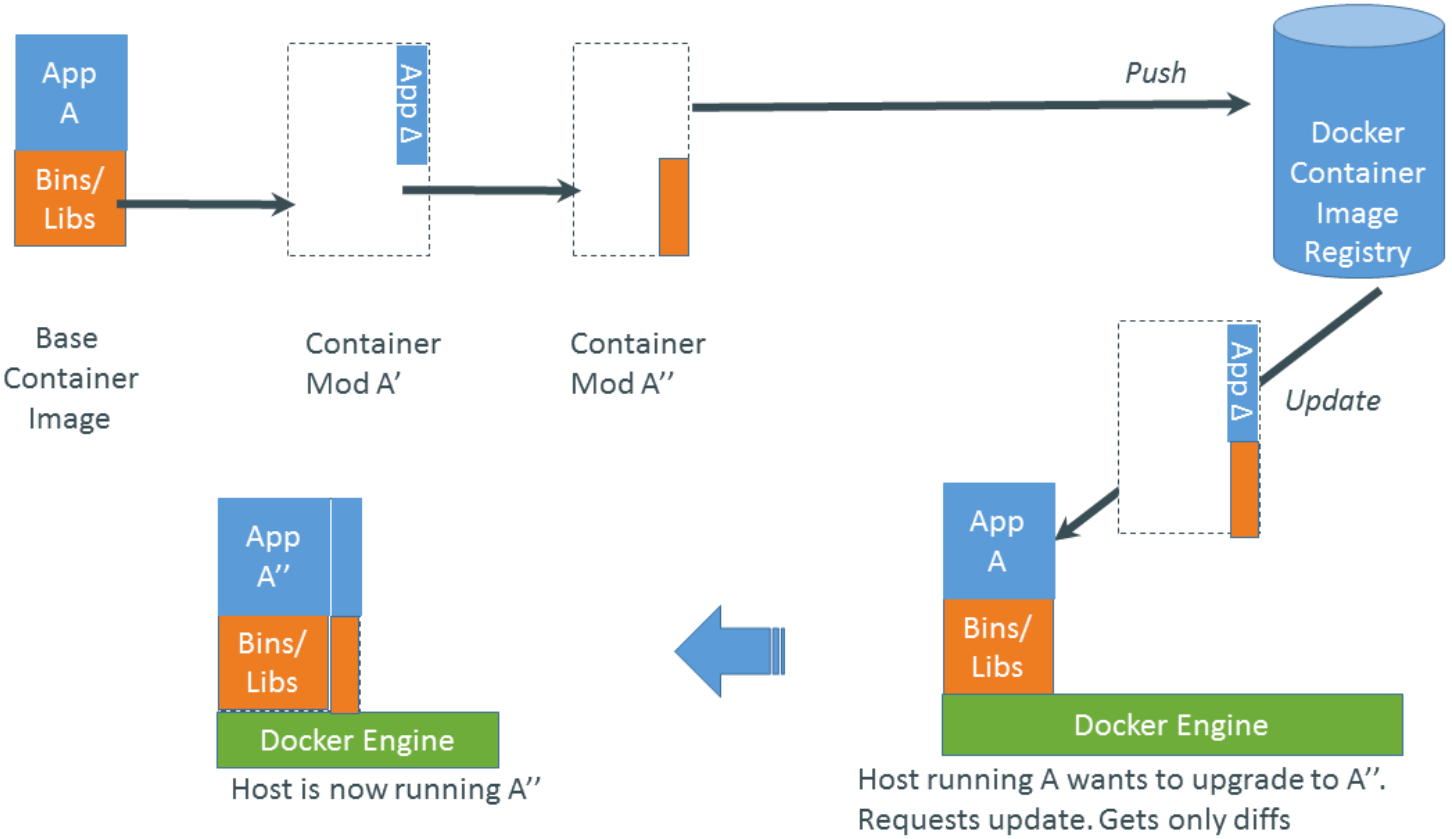




Docker Image Registry

- Registry containing docker images
  - Local registry on the same host
  - Docker Hub Registry: Globally shared
  - Private registry on docker.com

Changes and Updates



Threats of Containers

- Unlike VMs whose interface is hardware instructions, containers' interface is OS system calls
- More difficult to protect syscalls
  - Involve large amount of code in the OS
  - And there are many syscalls

Security Implications of Containers

TABLE 1. Threat model specifications for apps, containers, and host for the studied use cases. *'Semi'* refers to semi-honest. Apps in semi-honest/malicious containers can be semi-honest or malicious too.

Use Case	Apps can be			Containers can be			Host can be		
	honest	semi	malicious	honest	semi	malicious	honest	semi	malicious
(I) Protect container from applications	✓	✓	✓	✓	-	-	✓	-	-
(II) Inter-container protection	✓	-	-	✓	✓	✓	✓	-	-
(III) Protect host from containers	✓	✓	✓	✓	✓	✓	✓	-	-
(IV) Protect containers from host	✓	-	-	✓	-	-	✓	✓	✓

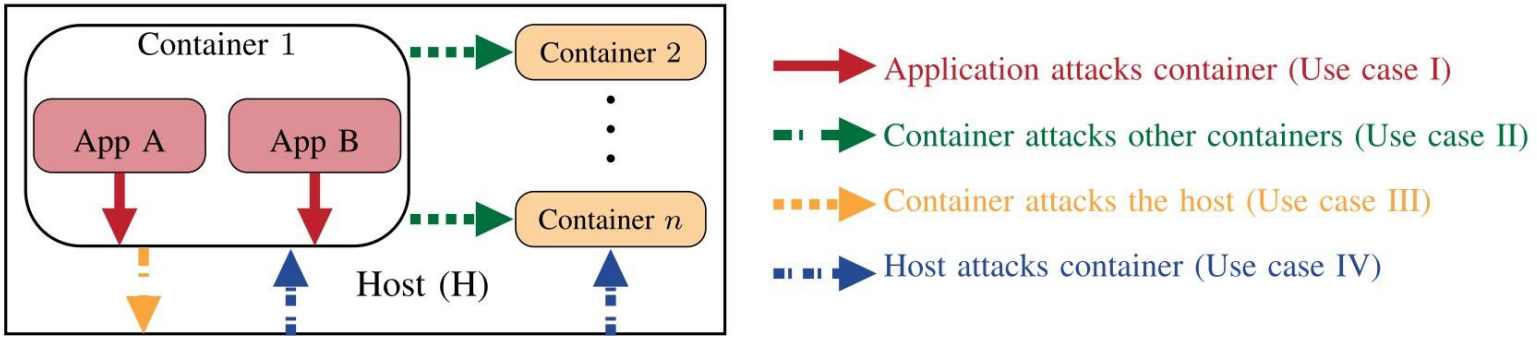


FIGURE 3. Overview of security protection requirements in containers.

Source: S. Sultan et al.: Container Security: Issues, Challenges, and the Road Ahead

CONTAINER HOMEWORK 4:

1. Define a container in one sentence and contrast it with a virtual machine in one sentence.
2. List four resources that Linux namespaces can isolate and give a one-line example for each.
3. What problem do cgroups solve? Give two concrete limits you could set with cgroups.
4. Explain why containers are called “lightweight.” Mention CPU, memory, and network overheads.
5. “Build once, run anywhere.” State the practical caveat to this claim from the slides and explain it.
6. Name the three main Docker Engine components and state the responsibility of each.
7. Multiple choice: Which statement about images is true?
   A) Images include a kernel., B) Images are read-only layered filesystems., C) Images always change at runtime., D) Images are the same thing as containers.
8. Short answer: What is the container writable layer used for, and what happens to it when the container is removed?
9. Describe the role of a Docker registry. Give one example of a public registry and one reason to use a private registry.
10. Show the single Docker command you would use to: (a) list running containers; (b) start a new container from image nginx:latest and publish port 8080 on the host to 80 in the container.
11. Layers: Suppose your Dockerfile installs a base OS, then Python, then applies a small security patch. Explain how layers and caching make the second build faster after you only change application code.
12. Admin vs. developer perspective: give two benefits containers provide to developers and two to administrators, as stated in the slides.

LAB Assignment

Assignment: “Hello, Docker—From Image to Container”

Learning goals

1. Build a Docker image from a Dockerfile.
2. Run a container from that image and expose a port.
3. Use environment variables for configuration.
4. Persist data with a volume (so it survives container removal).
5. Apply resource limits (CPU/RAM) and explain why they matter.
6. Explain image vs. container in one paragraph.

Questions?

Thank you!